

Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	01 October 2026
Team ID	SWTID-2026-7059
Project Name	FitBuddy
Maximum Marks	4 Marks

Technical Architecture:

The Deliverable shall include the architectural diagram below and the information as per Table 1 & Table 2.

Project: FitBuddy – AI-Powered Personalized Fitness Coach

System Architecture Diagram <pre> [User Browser] Form POST (name, age, wt, goal, intensity) ▼ [FastAPI Backend - routes.py] ─▶ Pydantic Validation (schemas.py) ─▶ Gemini Pro API → 7-Day Workout Plan ─▶ Gemini Flash API → Nutrition Tip └─▶ SQLite DB (save_user / save_plan) ▼ [Jinja2 Template - result.html] Display Plan + Tip + Feedback Form ▼ [User Submits Feedback] ─▶ Gemini Pro → Updated Plan → SQLite ▼ </pre>	Architecture Guidelines (FitBuddy): • All application logic implemented in FastAPI (Python) • Local development via Uvicorn; Cloud via Vercel Serverless • External API: Google Gemini Pro (workout plan generation) • External API: Google Gemini Flash (nutrition tips) • Data storage: SQLite (local) / /tmp/fitbuddy.db (Vercel) • Interface to Gemini AI models via google-genai SDK • Frontend: Jinja2 HTML templates (no separate JS framework)
--	--

[Vercel Serverless – api/index.py]	
---	--

Table-1: Component Details

S.No	Component	Description	Technology
1	User Interface	Web-based input form for user profile (name, age, weight, goal, intensity). Displays 7-day workout plan and nutrition tip. Includes feedback form and admin dashboard.	HTML5, CSS3 (Glassmorphism), Vanilla JS, Jinja2 Templates
2	Application Logic-1	Handles HTTP routing, form parsing, input validation, and HTML responses for all web pages.	FastAPI ≥ 0.115.0, Python 3.11+
3	Application Logic-2	Generates personalized 7-day workout plans by calling Gemini Pro with user profile data. Implements model fallback chain for reliability.	google-genai SDK ≥ 1.0.0, Gemini Pro (gemini-3.1-pro-preview)
4	Application Logic-3	Generates nutrition and recovery tips quickly using Gemini Flash. Provides fallback tips from static dictionary when API is unavailable.	Gemini Flash (gemini-3-flash-preview), Python
5	Database	Local SQLite database storing user profiles (id, name, age, weight, goal, intensity) and workout plans (original_plan, updated_plan).	SQLite (fitbuddy.db), SQLAlchemy ORM ≥ 2.0.30
6	Cloud Database	On Vercel deployment, SQLite is stored in /tmp/fitbuddy.db (ephemeral). For persistent cloud storage, PostgreSQL (DATABASE_URL env var) can be configured.	SQLite (/tmp on Vercel), PostgreSQL (optional via DATABASE_URL)
7	File Storage	Static files (images, CSS) served from /static directory. Gym background image hosted locally.	FastAPI StaticFiles, Local filesystem

S.No	Component	Description	Technology
8	External API-1	Google Gemini Pro model API used for generating comprehensive 7-day personalized workout plans with structured daily routines.	Google Gemini Pro API (via google-genai SDK, GOOGLE_API_KEY)
9	External API-2	Google Gemini Flash model API used for fast generation of nutrition and recovery tips based on the user's fitness goal.	Google Gemini Flash API (via google-genai SDK)
10	Machine Learning Model	Large Language Model (LLM) accessed via Gemini API. Generates structured text-based fitness plans and tips. No locally-trained ML model; uses Google's hosted Gemini LLM.	Google Gemini Pro / Flash LLM (hosted, via API key)
11	Infrastructure (Server / Cloud)	Local development: Uvicorn ASGI server. Cloud production: Vercel Serverless Python functions (api/index.py as the entry point, maxLambdaSize 15MB).	Uvicorn ≥ 0.30.0 (local), Vercel v2 Serverless (cloud)

Table-2: Application Characteristics

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	The entire application is built using open-source frameworks and libraries. FastAPI, SQLAlchemy, Pydantic, Uvicorn, and Jinja2 are all MIT/BSD licensed open-source Python packages.	FastAPI, SQLAlchemy, Pydantic, Jinja2, Uvicorn, python-dotenv
2	Security Implementations	API key (GOOGLE_API_KEY) stored in .env file and never exposed in code or HTML responses. Input validation via Pydantic schemas prevents injection attacks. Environment	python-dotenv, Pydantic validation, .env file, .gitignore

S.No	Characteristics	Description	Technology
		variable separation for local vs. cloud.	
3	Scalable Architecture	Deployed on Vercel serverless which auto-scales with demand. Stateless HTTP request handling. Database URL is configurable via environment variable to support scaling to cloud databases.	Vercel Serverless, DATABASE_URL env var, SQLAlchemy ORM
4	Availability	Gemini model fallback chain ensures availability even when primary model is unavailable (tries ≥ 3 models). Static nutrition tips serve as offline fallback when AI API is down.	Gemini model chain (gemini-3.1-pro-preview $\rightarrow$ gemini-pro-latest $\rightarrow$ gemini-flash), Python fallback dict
5	Performance	Gemini Flash model used for nutrition tips (faster, lower cost). Gemini Pro used for detailed workout plans. Caches last-working model to avoid retry overhead. UI loading state shown during AI generation.	Gemini Flash (low latency), Model caching (_working dict), Async-compatible FastAPI

References:

1. [Google Gemini API Documentation – https://ai.google.dev/gemini-api/docs](https://ai.google.dev/gemini-api/docs)
2. [FastAPI Official Documentation – https://fastapi.tiangolo.com/](https://fastapi.tiangolo.com/)
3. [SQLAlchemy ORM Documentation – https://docs.sqlalchemy.org/en/20/](https://docs.sqlalchemy.org/en/20/)
4. [Vercel Python Serverless Guide – https://vercel.com/docs/functions/runtimes/python](https://vercel.com/docs/functions/runtimes/python)
5. [Pydantic v2 Documentation – https://docs.pydantic.dev/latest/](https://docs.pydantic.dev/latest/)